A Fast Wait-Free Solution to Read-Reclaim Races in Reference Counting

Conteo de referencias concurrente, *wait-free* y sin reclamación diferida

Ivo Gabe de Wolff Daniel Anderson Gabriele K. Keller Aleksei Seletskiy

Euro-Par 2024 · LNCS 14803, pp. 103–118

Presentación del artículo

Contenido

1. Motivación y problema
2. Trabajo previo
3. Idea: conteo con pesos
4. Algoritmos load / store
5. Aumentar el peso sin esperas
6. Corrección y complejidad
7. Extensiones y límites
8. Evaluación experimental

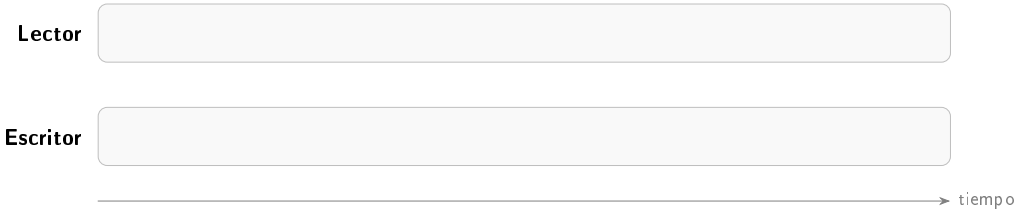
Motivación: conteo de referencias

- Gestión de memoria segura y simple: cada objeto lleva un contador de referencias (*refcount*).
- Muy usado: `shared_ptr` en C++, `Arc` en Rust, implícito en Swift.
- El caso difícil: una variable mutable compartida que contiene un puntero con conteo: `atomic<shared_ptr<T>>`.
- Operaciones: load (leer y obtener una referencia propia) y store (reemplazar el puntero).

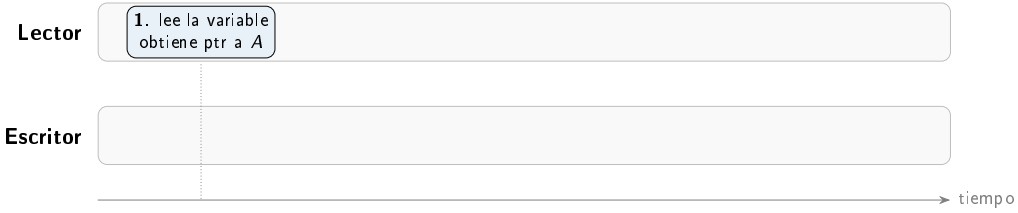
Pregunta del artículo

¿Se puede implementar `atomic<shared_ptr>` wait-free, rápido y sin sobrecoste de memoria?

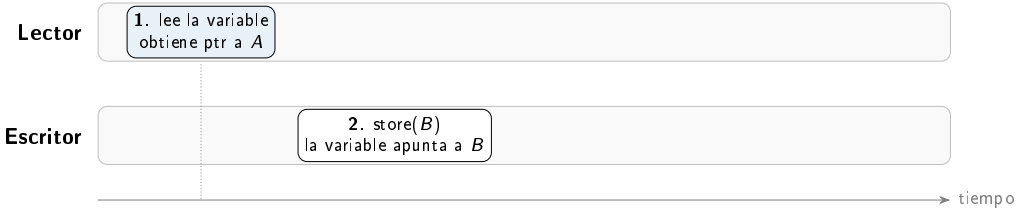
El problema: carrera *read-reclaim*



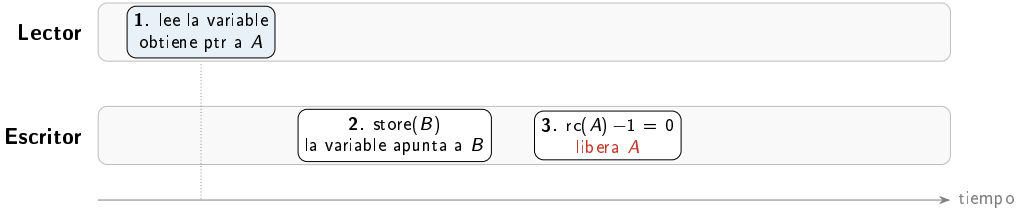
El problema: carrera *read-reclaim*



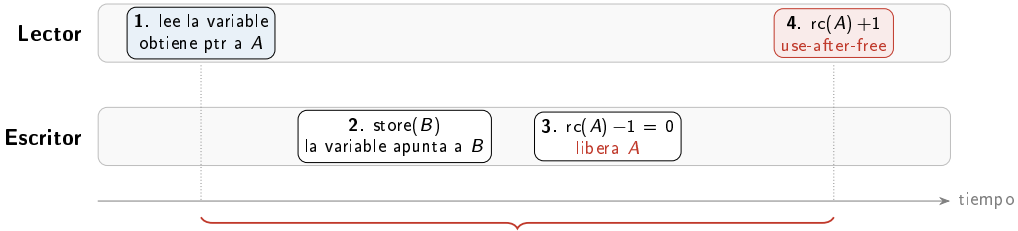
El problema: carrera *read-reclaim*



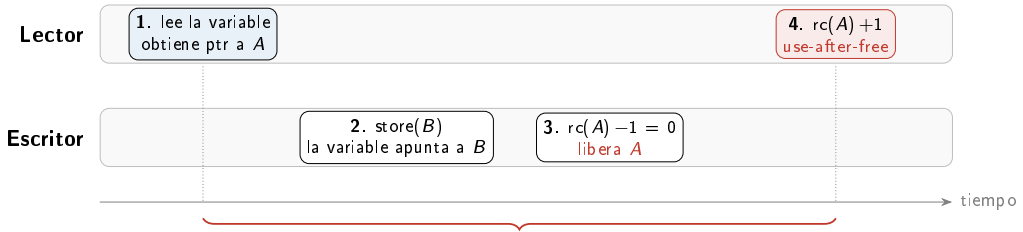
El problema: carrera *read-reclaim*



El problema: carrera *read-reclaim*



El problema: carrera *read-reclaim*



No existe instrucción que lea la variable *y* modifique el refcount de otro objeto a la vez (DCAS no existe en hardware actual).

Soluciones existentes

Enfoque	Wait-free	Sin sobrecoste de memoria	Hardware
Candados (libstdc++)	✗	✓	✓
Lock-free clásico (Valois; Michael & Scott)	✗	✗	✓
Detlefs et al. (DCAS)	✗	✓	✗
Reclamación diferida (HP, épocas, CDRC)	—	✗	✓
Sundell	✓	✓	✓ ($\Theta(P)$ po
Split reference count (Williams, Folly)	✗	✓	✓
Este trabajo	✓	✓	✓

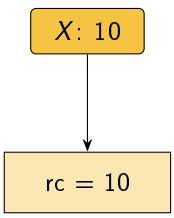
store en $O(1)$; load casi siempre $O(1)$ y $O(P)$ en el peor caso.

Contribuciones

1. Estrategia para `atomic<shared_ptr>` wait-free en store, load y swap, con hardware actual.
2. load: normalmente una sola instrucción atómica; peor caso $O(P)$.
   store: tiempo constante.
3. Nueva explicación del *split reference count* como caso particular del conteo de referencias con pesos.
4. Soporte para compare-and-swap y punteros *aliased* (estos, solo lock-free).
5. Implementación en C++ y evaluación frente a cuatro alternativas.

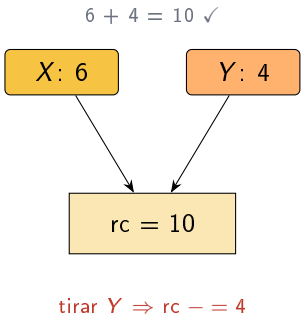
Idea clave: conteo de referencias con pesos

- Bevan; Watson & Watson (1987).
- Cada referencia tiene un peso ≥ 1.
- Invariante: rc(obj) $= \sum$ pesos.
- Copiar = partir el peso. ¡No toca el refcount!
- Liberar = restar su peso al refcount.



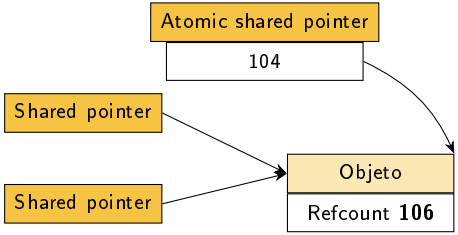
Idea clave: conteo de referencias con pesos

- Bevan; Watson & Watson (1987).
- Cada referencia tiene un peso ≥ 1.
- Invariante: rc(obj) $= \sum$ pesos.
- Copiar = partir el peso. ¡No toca el refcount!
- Liberar = restar su peso al refcount.



Pesos en el puntero atómico compartido

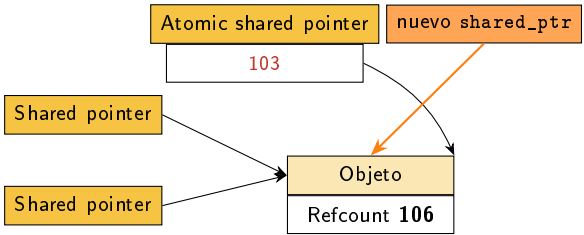
- Al crear el puntero atómico: peso $= C \gg P$ y se suma C al refcount.
- load resta 1 al peso y entrega un `shared_ptr` de peso 1.
- El refcount del objeto no cambia: ya estaba *prepagado*.



$104 + 1 + 1 = 106$

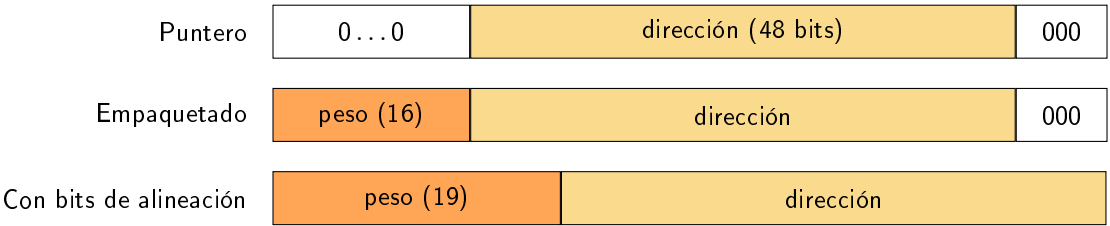
Pesos en el puntero atómico compartido

- Al crear el puntero atómico: peso $= C \gg P$ y se suma C al refcount.
- load resta 1 al peso y entrega un `shared_ptr` de peso 1.
- El refcount del objeto no cambia: ya estaba *prepagado*.



$103 + 1 + 1 + 1 = 106$ ✓

Empaquetar puntero y peso en una palabra



- CPUs de 64 bits usan 48 bits de dirección $\Rightarrow$ 16 bits libres.
- Con los bits de alineación: $C = 2^{19} - 1$. En los experimentos: $C = 2^{19}$, $P = 2^{16}$.
- load = un único fetch-and-add sobre la palabra: decrementa el peso y devuelve el puntero.

Algoritmo 1: store y load

```
1: function store(this, new)
2:     Incrementar rc(new) en C
3:     (p, w) ← unpack(swap(this, pack(new, C)))
4:     Decrementar rc(p) en w

5: function load(this)
6:     (p, w) ← unpack(fetch_add(this, −pack(null, 1)))
7:     if w − 1 < P then
8:         increase_weight(this, p, w − 1)
9:     return shared_ptr(p)
```

Dos operaciones, cada una con una instrucción atómica central.

Algoritmo 1: store y load

```
1: function store(this, new)
2:     Incrementar rc(new) en C
3:     (p, w) ← unpack(swap(this, pack(new, C)))
4:     Decrementar rc(p) en w

5: function load(this)
6:     (p, w) ← unpack(fetch_add(this, −pack(null, 1)))
7:     if w − 1 < P then
8:         increase_weight(this, p, w − 1)
9:     return shared_ptr(p)
```

store: prepaga el objeto nuevo con C, lo instala con un *swap* atómico y resta al viejo solo los w tickets *no usados*.

Algoritmo 1: store y load

```
1: function store(this, new)
2:     Incrementar rc(new) en C
3:     (p, w) ← unpack(swap(this, pack(new, C)))
4:     Decrementar rc(p) en w

5: function load(this)
6:     (p, w) ← unpack(fetch_add(this, −pack(null, 1)))
7:     if w − 1 < P then
8:         increase_weight(this, p, w − 1)
9:     return shared_ptr(p)
```

load: leer y «apuntarse» es un solo fetch-and-add: no hay ventana para la carrera.

Algoritmo 1: store y load

```
1: function store(this, new)
2:     Incrementar rc(new) en C
3:     (p, w) ← unpack(swap(this, pack(new, C)))
4:     Decrementar rc(p) en w

5: function load(this)
6:     (p, w) ← unpack(fetch_add(this, −pack(null, 1)))
7:     if w − 1 < P then
8:         increase_weight(this, p, w − 1)
9:     return shared_ptr(p)
```

Si el peso cae por debajo de P, este hilo debe recargarlo antes de continuar.

Aumentar el peso: requisitos

Problema

Recargar el peso no puede ser un fetch-and-add: hay que comprobar que el puntero no cambió ni fue ya recargado $\Rightarrow$ bucle de compare-and-swap.

R-1

$C \geq 2P$: el peso máximo es al menos el doble del número de hilos.

R-2

El peso solo cambia de dos formas: baja de uno en uno o salta por encima de P.

Consecuencia: si un CAS falla porque el peso *subió*, el peso estuvo por encima de $P \Rightarrow$ otro hilo ya recargó $\Rightarrow$ salida temprana.

Algoritmo 2: increase_weight (wait-free)

```
1:  function increase_weight(value, ptr, weight)
2:      Incrementar rc(ptr) en C − weight
3:      cur ← weight
4:      loop
5:          cas ← CAS(value, (ptr, cur),
    (ptr, cur + C − weight))
6:          if cas.success then return
7:          (p′, w′) ← unpack(cas.original)
8:          if p′ ≠ ptr or w′ > cur then break
9:          cur ← w′
10:     Decrementar rc(ptr) en C − weight
```

Bucle CAS con tres salidas posibles.

Algoritmo 2: increase_weight (wait-free)

```
1:  function increase_weight(value, ptr, weight)
2:      Incrementar rc(ptr) en C − weight
3:      cur ← weight
4:      loop
5:          cas ← CAS(value, (ptr, cur),
    (ptr, cur + C − weight))
6:          if cas.success then return
7:          (p′, w′) ← unpack(cas.original)
8:          if p′ ≠ ptr or w′ > cur then break
9:          cur ← w′
10:     Decrementar rc(ptr) en C − weight
```

Reservar antes de publicar:
nunca hay pesos en el puntero sin respaldo en el refcount.

Algoritmo 2: increase_weight (wait-free)

```
1:  function increase_weight(value, ptr, weight)
2:      Incrementar rc(ptr) en C − weight
3:      cur ← weight
4:      loop
5:          cas ← CAS(value, (ptr, cur),
    (ptr, cur + C − weight))
6:          if cas.success then return
7:          (p′, w′) ← unpack(cas.original)
8:          if p′ ≠ ptr or w′ > cur then break
9:          cur ← w′
10:     Decrementar rc(ptr) en C − weight
```

Instala el nuevo peso solo si nada cambió.

Algoritmo 2: increase_weight (wait-free)

```
1:  function increase_weight(value, ptr, weight)
2:      Incrementar rc(ptr) en C − weight
3:      cur ← weight
4:      loop
5:          cas ← CAS(value, (ptr, cur),
    (ptr, cur + C − weight))
6:          if cas.success then return
7:          (p′, w′) ← unpack(cas.original)
8:          if p′ ≠ ptr or w′ > cur then break
9:          cur ← w′
10:     Decrementar rc(ptr) en C − weight
```

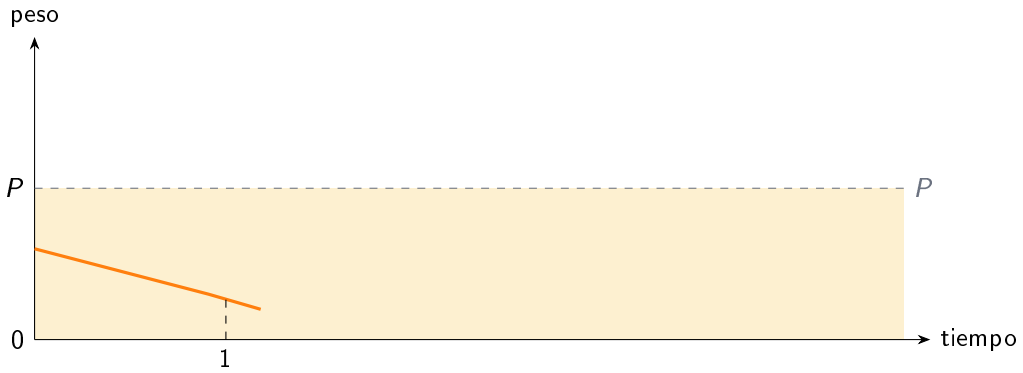
Salida temprana: otro objeto, o el peso subió (R-2) $\Rightarrow$ ya recargado.

Algoritmo 2: increase_weight (wait-free)

```
1:  function increase_weight(value, ptr, weight)
2:      Incrementar rc(ptr) en C − weight
3:      cur ← weight
4:      loop
5:          cas ← CAS(value, (ptr, cur),
    (ptr, cur + C − weight))
6:          if cas.success then return
7:          (p′, w′) ← unpack(cas.original)
8:          if p′ ≠ ptr or w′ > cur then break
9:          cur ← w′
10:     Decrementar rc(ptr) en C − weight
```

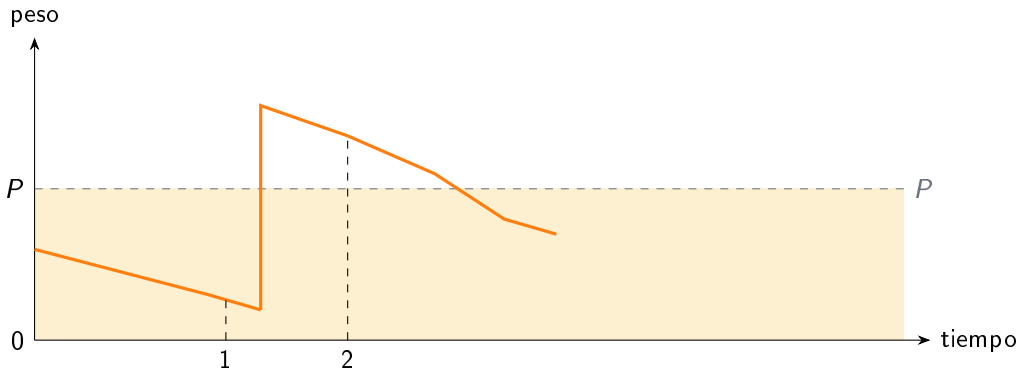
Si el peso bajó, reintenta. Los valores esperados decrecen estrictamente: $\leq P$ iteraciones.

El peso a lo largo del tiempo



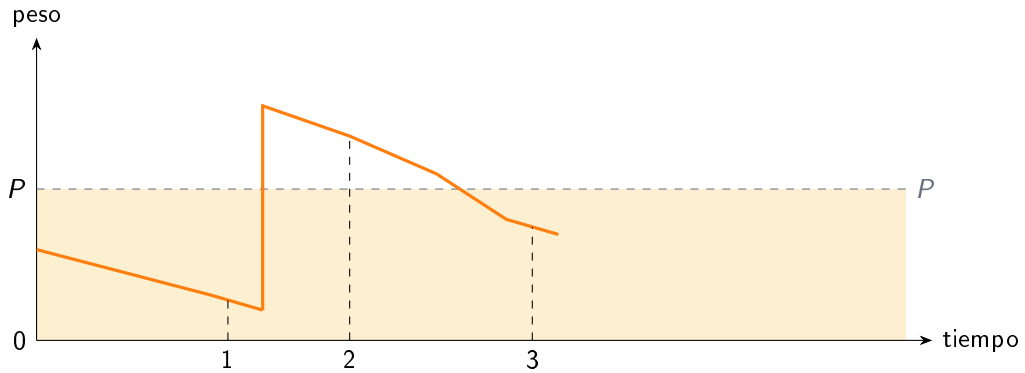
1. Un hilo hace load, deja el peso bajo P y llama a increase_weight.

El peso a lo largo del tiempo



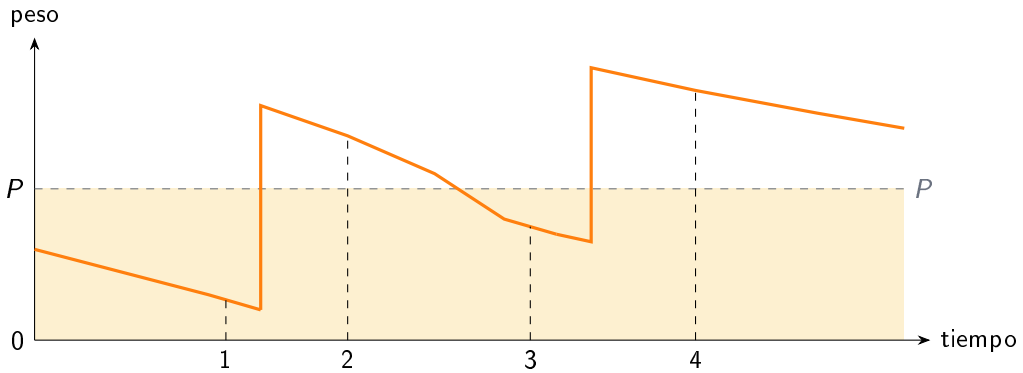
2. Otro hilo recarga primero: el peso supera P.

El peso a lo largo del tiempo



3. Nuestro hilo ve un CAS fallido con peso mayor al esperado (aunque bajo P) $\Rightarrow$ sale: ya hubo recarga.

El peso a lo largo del tiempo



4. Los hilos que bajaron el peso después de 2 son responsables de la siguiente recarga.

Corrección: el peso nunca es negativo

- **Ofensa**: un hilo decrementa el peso cuando este es $\leq P$.
- O: multiconjunto de hilos ofensores; se vacía cuando el peso supera P.

Corrección: el peso nunca es negativo

- **Ofensa**: un hilo decrementa el peso cuando este es $\leq P$.
- O: multiconjunto de hilos ofensores; se vacía cuando el peso supera P.

Invariante 1: cada hilo está en O como mucho una vez

Tras ofender, el hilo debe salir de increase_weight; las tres salidas implican que el peso estuvo $> P \Rightarrow O$ se vació.

Corrección: el peso nunca es negativo

- **Ofensa**: un hilo decrementa el peso cuando este es $\leq P$.
- O: multiconjunto de hilos ofensores; se vacía cuando el peso supera P.

Invariante 1: cada hilo está en O como mucho una vez

Tras ofender, el hilo debe salir de increase_weight; las tres salidas implican que el peso estuvo $> P \Rightarrow O$ se vació.

Invariante 2: peso $\geq P - |O|$

Por R-2, desde el último vaciado el peso solo baja de uno en uno, y cada bajada bajo P añade un hilo a O.

Conclusión

$|O| \leq P \;\Rightarrow\; \text{peso} \geq P - P = 0.$ Con 1 o con 2^{16} hilos.

Complejidad

Operación	Progreso	Coste
load	wait-free	casi siempre $O(1)$; peor caso O
store	wait-free	$O(1)$
swap	wait-free	$O(1)$
compare-and-swap	lock-free	—
punteros *aliased*	wait-free (rápido) / lock-free (fallback)	—

- En la práctica, el peor caso depende de los hilos que realmente usan esa variable, no de P.
- Con $C = 2^{19}$ y $P = 2^{16}$: $\approx 4{,}6 \times 10^5$ loads entre recargas.

Extensiones y limitaciones

Extensiones

- **null**: el peso puede desbordar sin alterar el puntero (bits altos).
- **Punteros débiles**: ortogonales; mismo esquema.
- **Compare-and-swap**: lock-free (falla si cambia el peso).
- **Aliased**: palabra de 128 bits; ruta rápida optimista + fallback lock-free.

Limitaciones

- Límite duro de P hilos.
- Requiere bits libres en los punteros (afecta tagging, LA57, ARM TBI/PAC).
- Cada load escribe en la variable: contención en lecturas.
- Sin *snapshot pointers*.
- Refcount de 64 bits recomendado.

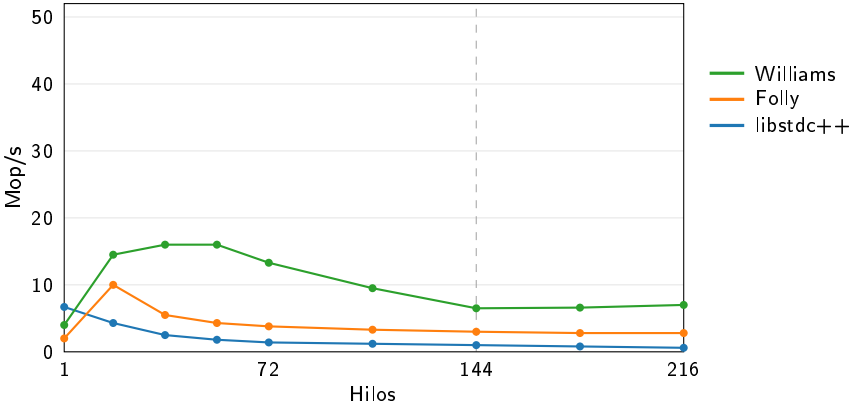
Configuración experimental

- **Throughput**: 4× Intel Xeon E7-8867, 72 núcleos / 144 hilos SMT, 1 TB.
- **Latencia**: Intel Xeon w7-3465X, 28 núcleos / 56 hilos, 128 GB.
- Pila concurrente: n pilas, u % de actualizaciones.
- $C = 2^{19}$, $P = 2^{16}$.

Implementaciones comparadas

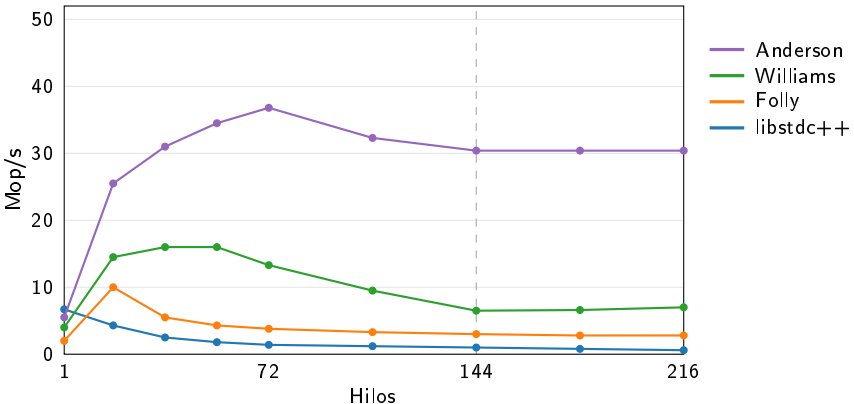
- **Nuestro** (wait-free)
- **libstdc++** (candados)
- **Williams** (split rc, lock-free)
- **Folly** (split rc, lock-free)
- **Anderson** (hazard pointers; pequeño sobrecoste de memoria)

Throughput: baja contención ($n = 10$, $u = 1\%$)



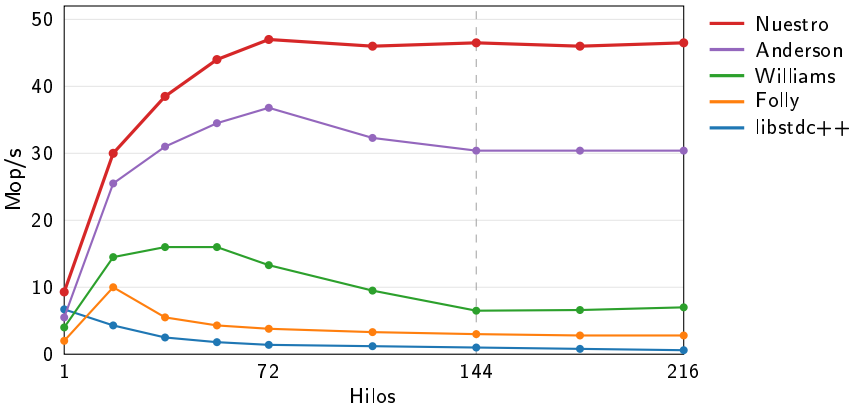
Los métodos sin sobrecoste de memoria colapsan al crecer los hilos.

Throughput: baja contención ($n = 10$, $u = 1\%$)



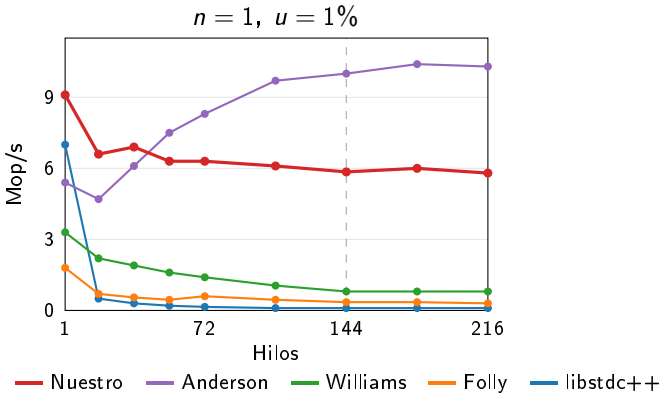
Anderson (reclamación diferida) escala mejor...

Throughput: baja contención ($n = 10$, $u = 1\%$)



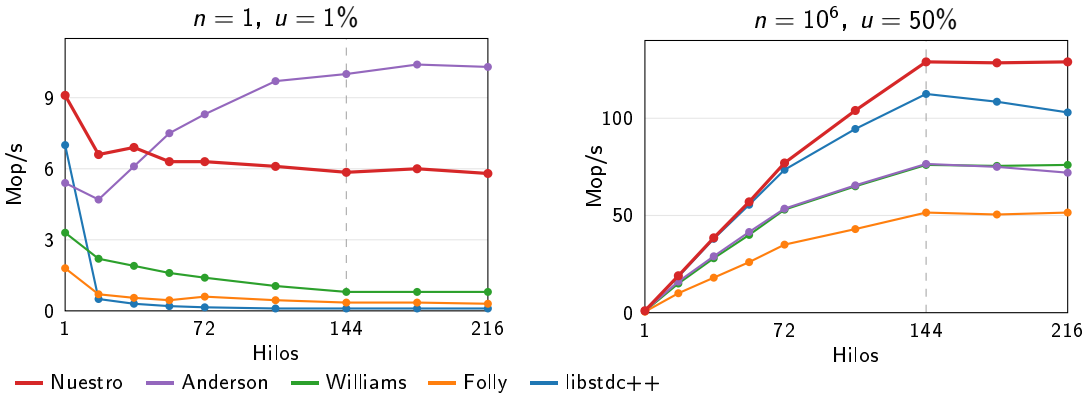
. . . pero el método propuesto llega a ≈47 Mop/s, ∼1,5× Anderson y ∼7× Williams.

Throughput: alta contención vs. muchas pilas



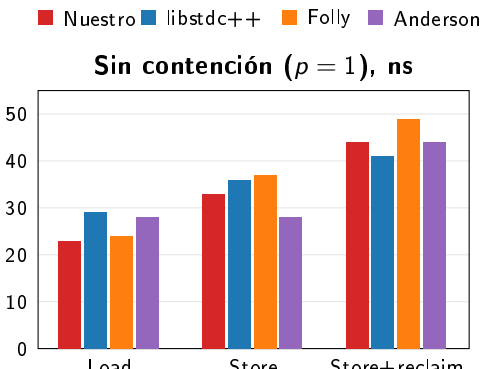
Nuestro es el mejor sin sobrecoste de memoria, pero con una sola pila Anderson gana: todas las lecturas escriben en la misma palabra.

Throughput: alta contención vs. muchas pilas



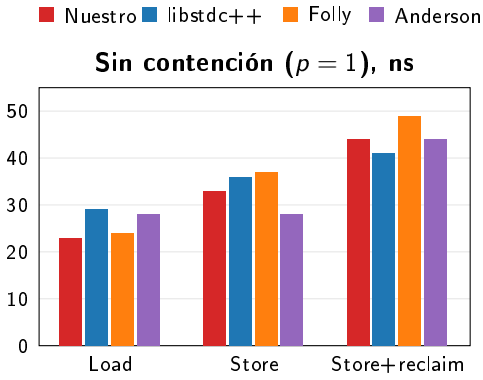
Con 10^6 pilas (poca contención) gana **Nuestro**: $\approx$129 Mop/s frente a 113 (libstdc++) y 77 (Anderson).

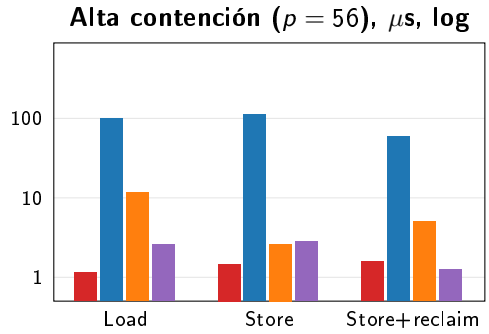
Latencia (mediana)



Sin contención todos son similares; load más rápido con **Nuestro** (23 ns).

Latencia (mediana)

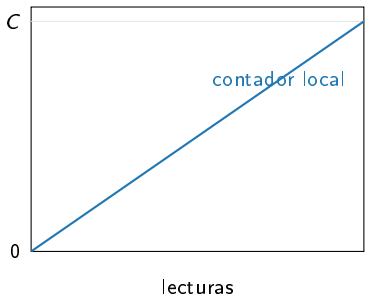




Con contención, load es $\sim$2,3$\times$ más rápido que Anderson y $\sim$87$\times$ que libstdc++.
Store+reclaim: Anderson ligeramente mejor.

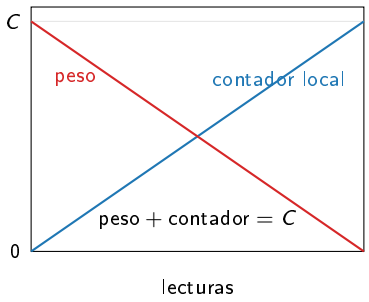
Relación con el *split reference count*

- Split rc: un contador local en el puntero que empieza en 0 y *sube* con cada lectura.
- Aquí: un peso que empieza en C y *baja*.
- Equivalencia: peso $= C -$ contador local.
- Diferencia clave: salida temprana en increase_weight $\Rightarrow$ wait-free.
- Folly: refcount de 32 bits, $C = 8192$, CAS en load; puede volverse bloqueante con $> C$ hilos.



Relación con el *split reference count*

- Split rc: un contador local en el puntero que empieza en 0 y *sube* con cada lectura.
- Aquí: un peso que empieza en C y *baja*.
- Equivalencia: peso $= C -$ contador local.
- Diferencia clave: salida temprana en increase_weight $\Rightarrow$ wait-free.
- Folly: refcount de 32 bits, $C = 8192$, CAS en load; puede volverse bloqueante con $> C$ hilos.



Conclusiones

- `atomic<shared_ptr>` puede ser wait-free sin reclamación diferida.
- La carrera read-reclaim se elimina asignando un peso al puntero atómico: leer y adquirir la referencia es una sola instrucción.
- R-1 ($C \geq 2P$) + R-2 + salida temprana $\Rightarrow$ recarga acotada por P.
- En la práctica: mayor throughput que las alternativas sin sobrecoste y latencia de load mínima.
- Límites: CAS solo lock-free, contención en lecturas de una misma variable, dependencia de bits libres.

Código y artefacto: Zenodo (doi:10.5281/zenodo.11775182) y github.com/ivogabe/wait-free-atomic-shared-pointer

¡Gracias!

Preguntas